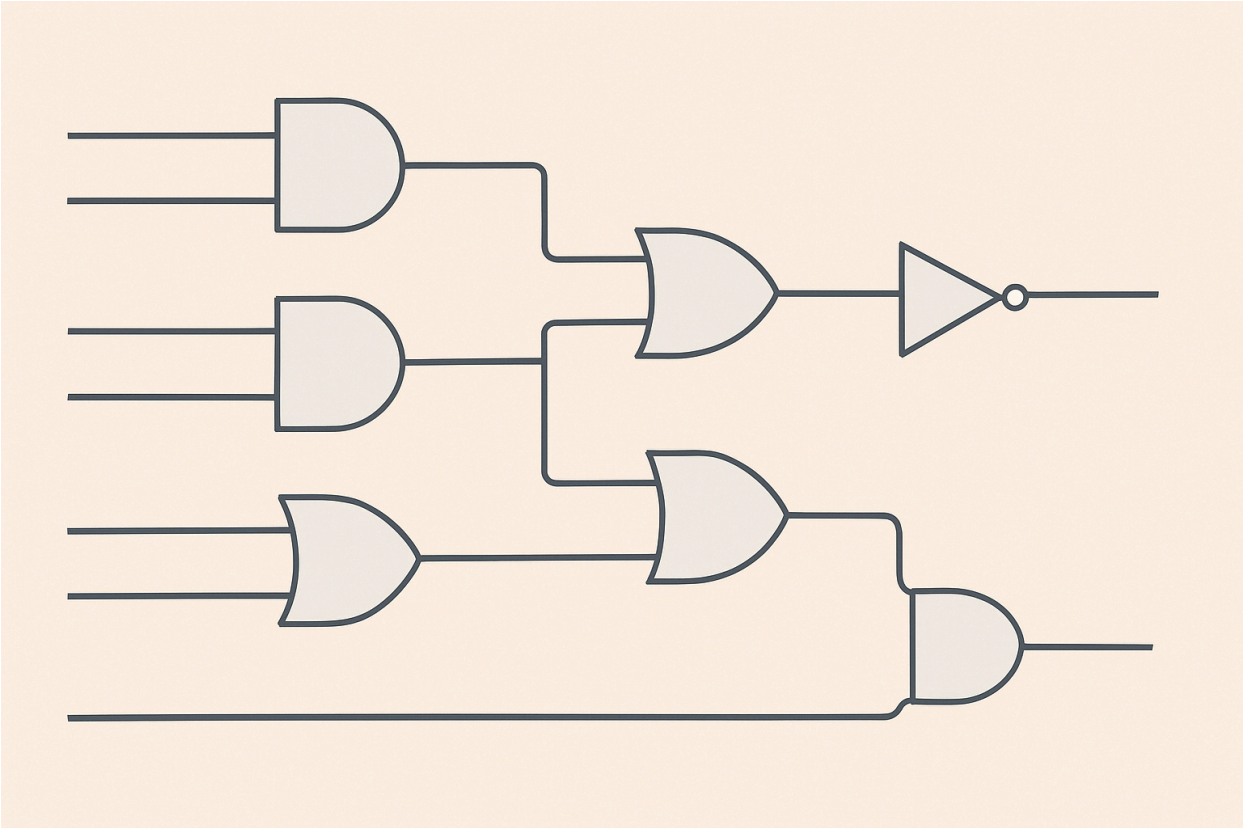


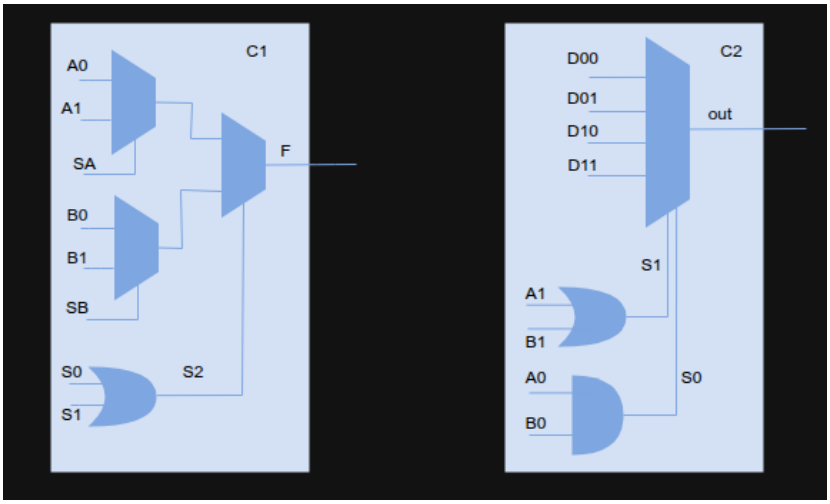
CA#3

Erfan Falahati - 810102491

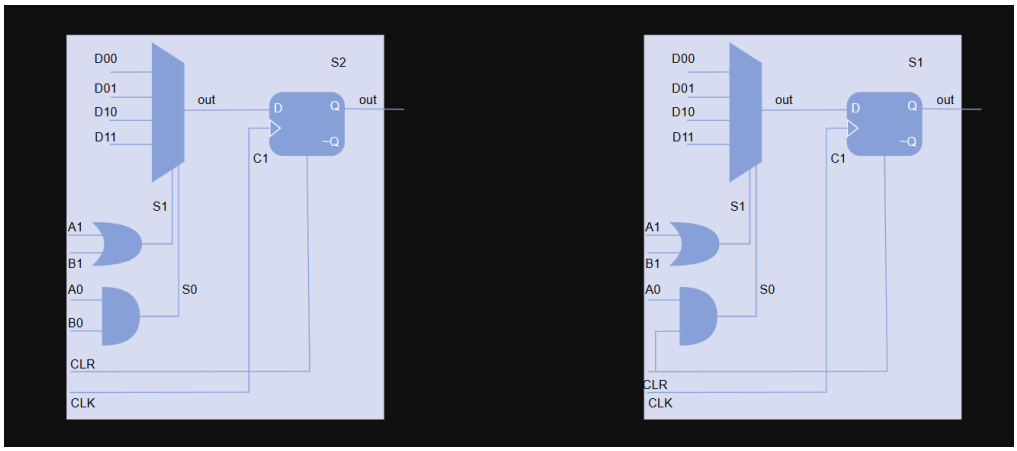
Sadegh Samadi - 810102588

Base Blocks

Combinational Blocks:

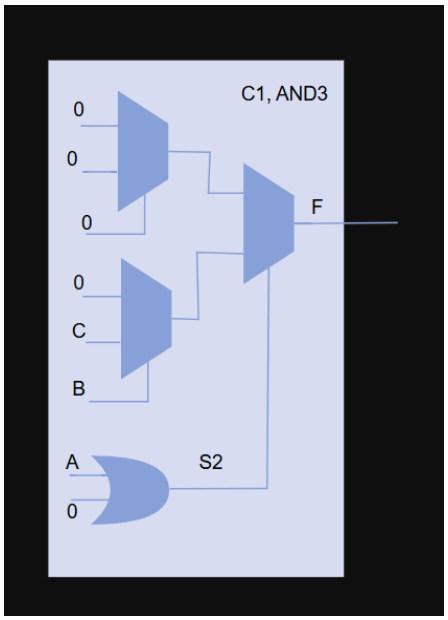


Sequential Blocks:



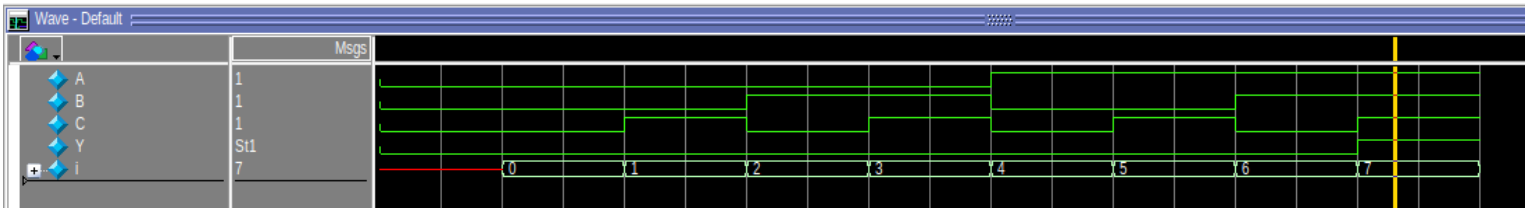
Base Gates

3 input AND:

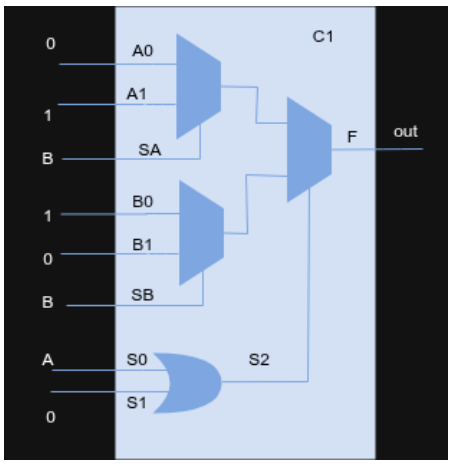


modules_code > TestBenches > AND3_TB.v

```
1   `timescale 1ns / 1ns
2
3   module AND3_tb;
4       reg A;
5       reg B;
6       reg C;
7       wire Y;
8       AND3 uut (
9           .A(A),
10          .B(B),
11          .C(C),
12          .Y(Y)
13      );
14
15      integer i;
16      initial begin
17          A = 0;
18          B = 0;
19          C = 0;
20          #10;
21          for (i = 0; i < 8; i = i + 1) begin
22              {A, B, C} = i;
23              #10;
24          end
25          $stop;
26      end
27
28  endmodule
```

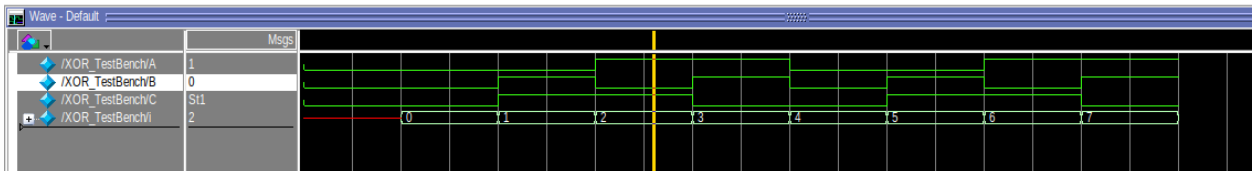


XOR:

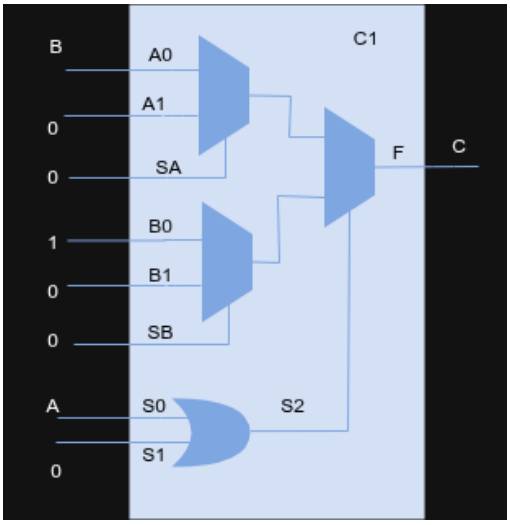


modules_code > TestBenches > xor_tb.v

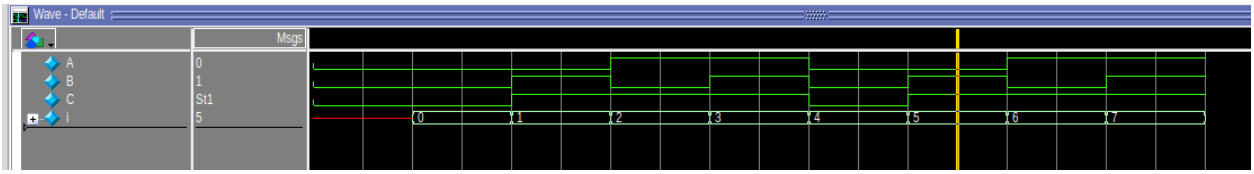
```
1   `timescale 1ns / 1ns
2
3   module XOR_TestBench;
4       reg A;
5       reg B;
6       wire C;
7       XOR uut (
8           .A(A),
9           .B(B),
10          .C(C)
11      );
12
13      integer i;
14      initial begin
15          A = 0;
16          B = 0;
17          #10;
18          for (i = 0; i < 8; i = i + 1) begin
19              {A, B} = i;
20              #10;
21          end
22          $stop;
23      end
24
25  endmodule
```



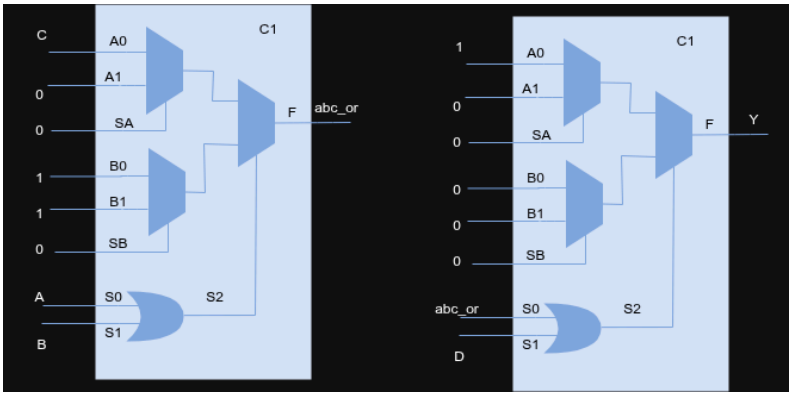
OR:



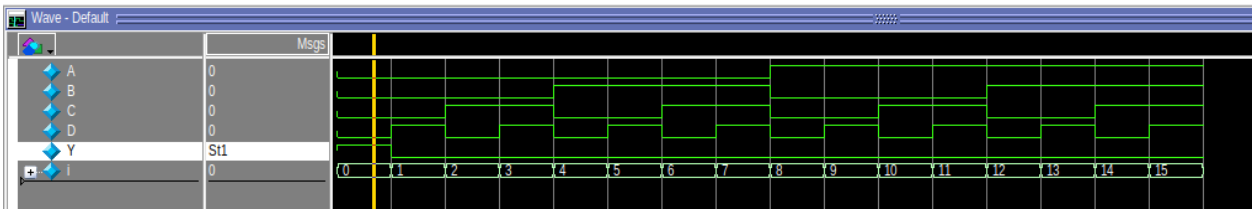

```
1   `timescale 1ns / 1ns
2
3   module OR_TestBench;
4       reg A;
5       reg B;
6       wire C;
7       OR uut (
8           .A(A),
9           .B(B),
10          .C(C)
11      );
12
13      integer i;
14      initial begin
15          A = 0;
16          B = 0;
17          #10;
18          for (i = 0; i < 8; i = i + 1) begin
19              {A, B} = i;
20              #10;
21          end
22          $stop;
23      end
24
25  endmodule
```



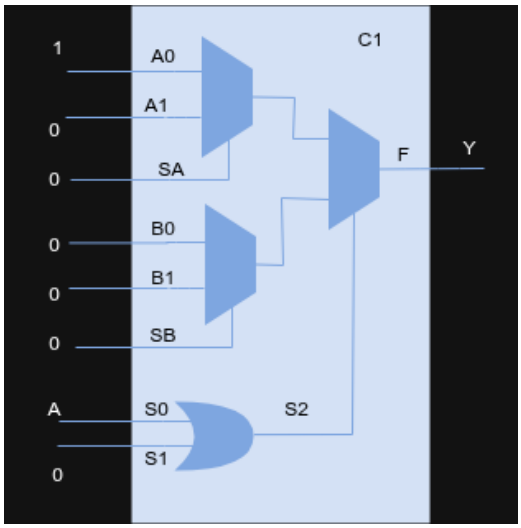
NOR 4 input:



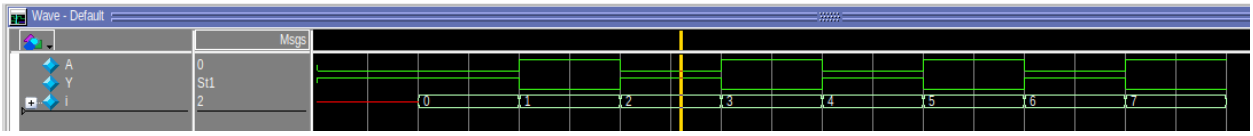

```
modules_code > TestBenches > nor_TB.v
1  `timescale 1ns / 1ns
2
3  module NOR4_tb;
4
5      reg A;
6      reg B;
7      reg C;
8      reg D;
9      wire Y;
10
11     NOR4 uut (
12         .A(A),
13         .B(B),
14         .C(C),
15         .D(D),
16         .Y(Y)
17     );
18
19     integer i;
20
21     initial begin
22         for (i = 0; i < 16; i = i + 1) begin
23             {A, B, C, D} = i[3:0];
24             #10;
25         end
26         $stop;
27     end
28
29 endmodule
```



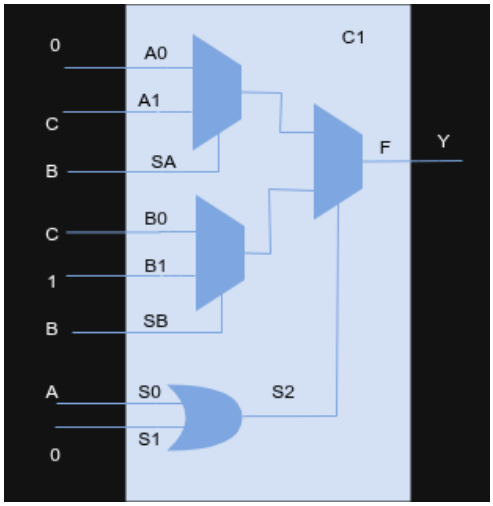
NOT:




```
1   `timescale 1ns / 1ns
2
3   module NOT_TestBench;
4       reg A;
5       wire Y;
6       NOT uut (
7           .A(A),
8           .Y(Y)
9       );
10
11      integer i;
12      initial begin
13          A = 0;
14          #10;
15          for (i = 0; i < 8; i = i + 1) begin
16              {A} = i;
17              #10;
18          end
19          $stop;
20      end
21
22  endmodule
```



MAJ:



modules_code > TestBenches > maj_tb.v

```verilog
`timescale 1ns/1ps

module MAJ_TB;

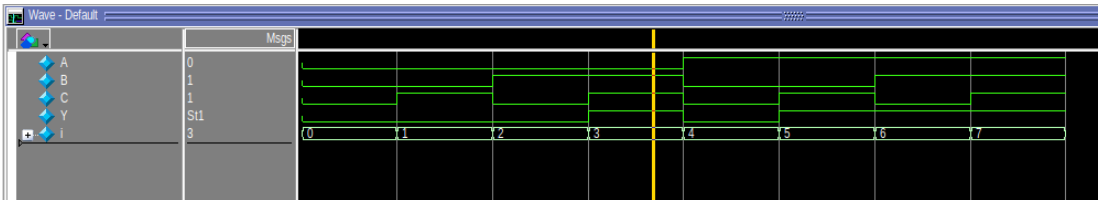
    reg A, B, C;
    wire Y;

    MAJ dut (
        .A(A),
        .B(B),
        .C(C),
        .Y(Y)
    );

    integer i;

    initial begin
        for (i = 0; i < 8; i = i + 1) begin
            {A, B, C} = i[2:0];
            #5;
        end
        $stop;
    end

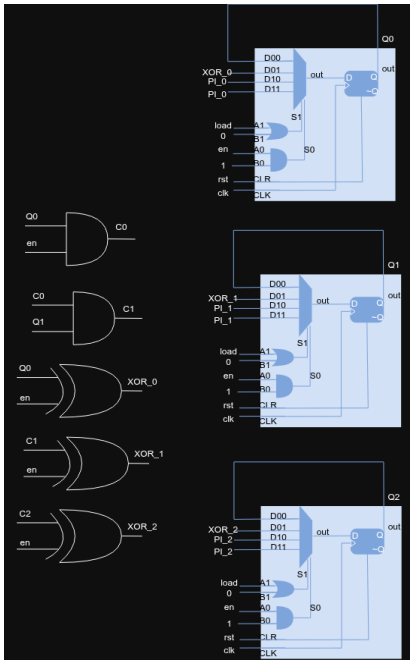
endmodule
```

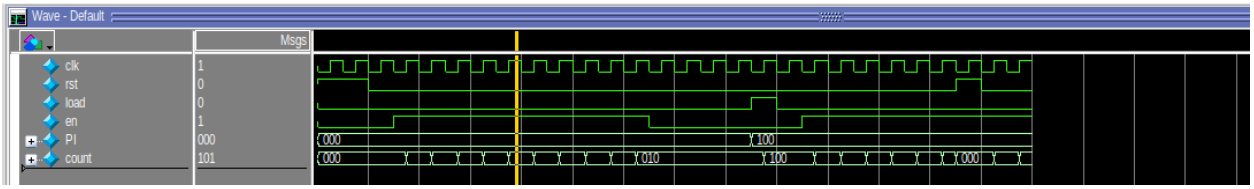


Base Modules

Counter 3 bit:

Testbench code in modules_code/TestBenches/Counter_TB.v .

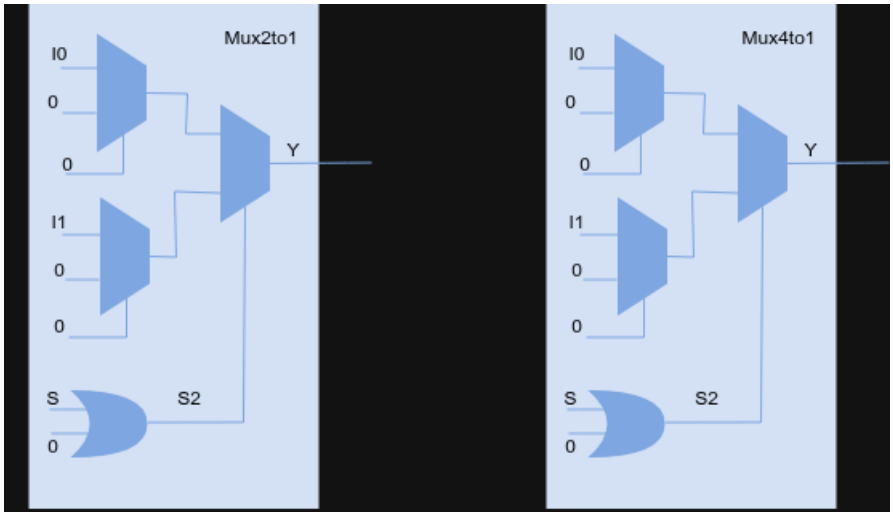




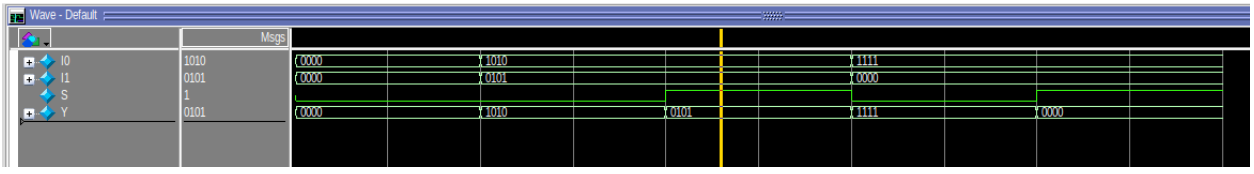
Counter 6 bit:

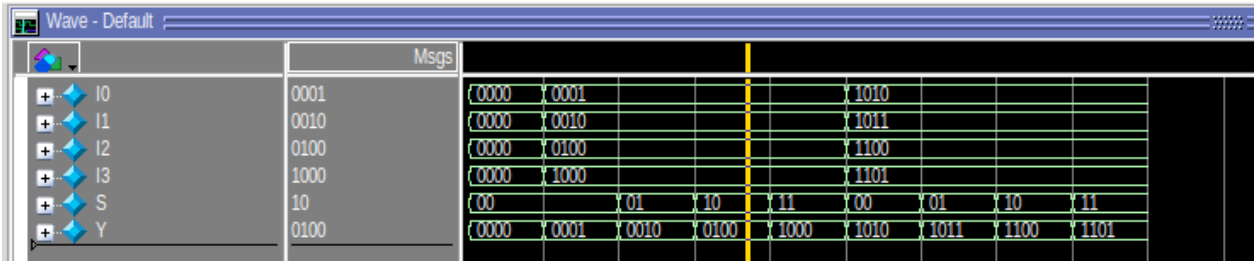
Similar to 3 bit counter

Mux:

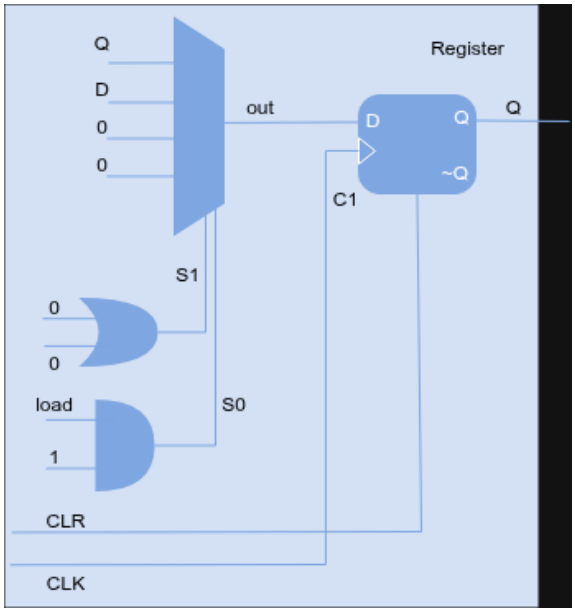


Testbenches code in modules_code/TestBenches/Mux21_TB.v & Mux41_TB.v .

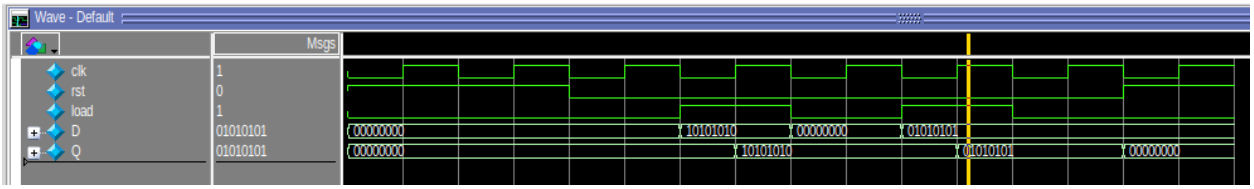




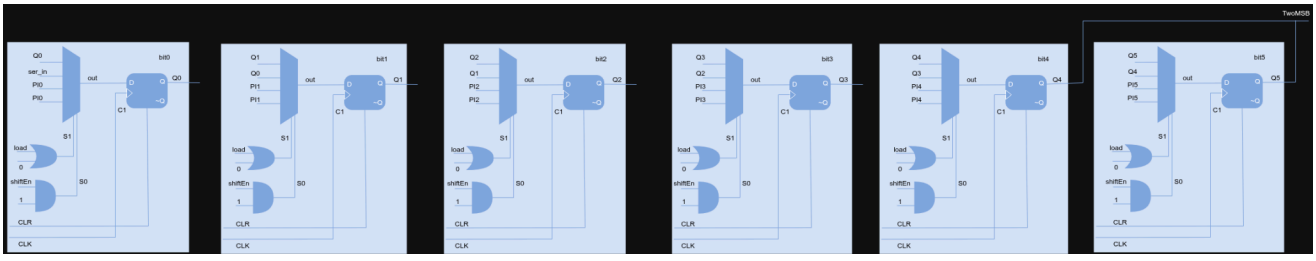
Register:



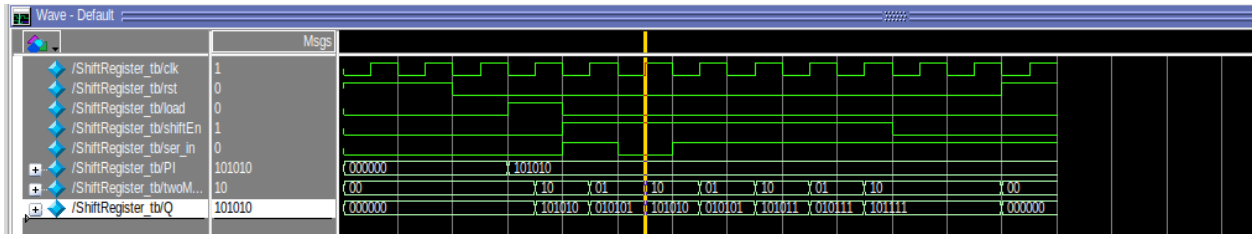
Testbench code in modules_code/TestBenches/Register_TB.v .



Shift Register:

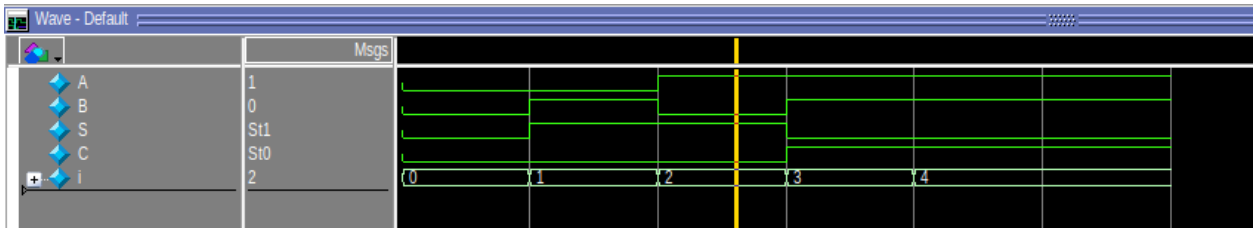


Testbenches code in modules_code/TestBenches/ShiftRegister_TB.v .



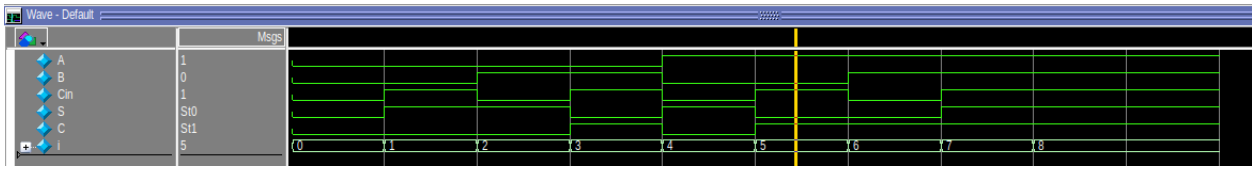
Half Adder:

Testbenches code in modules_code/TestBenches/HA_tb.v .



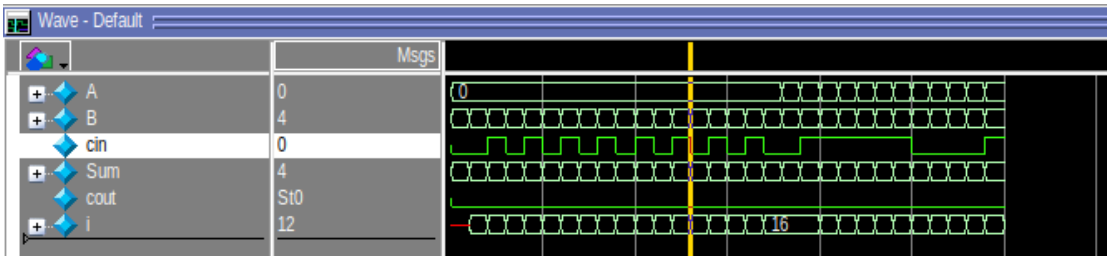
Full Adder:

Testbenches code in modules_code/TestBenches/FA_tb.v .



Adder:

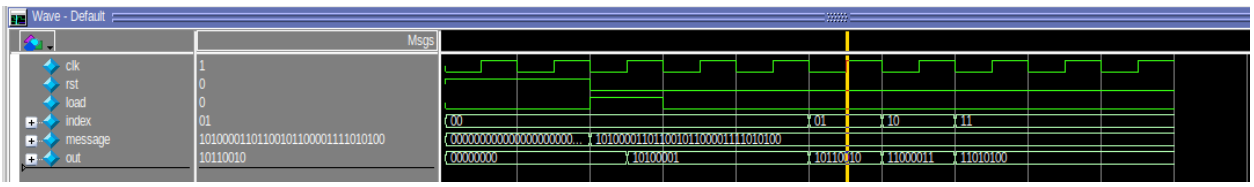
Testbenches code in modules_code/TestBenches/adder_tb.v .



Core Modules

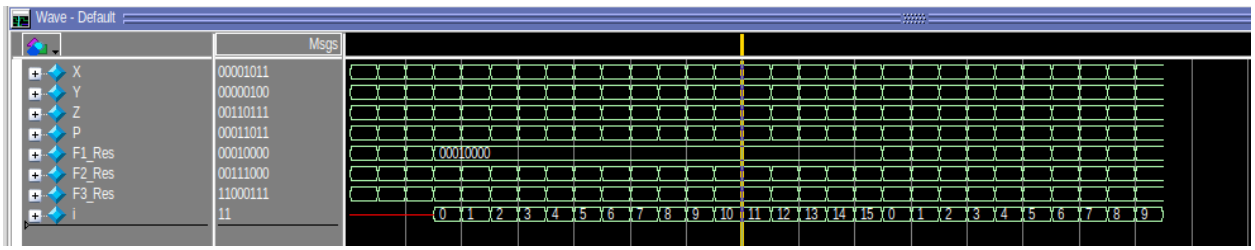
M File:

Testbenches code in modules_code/TestBenches/mfile_tb.v .

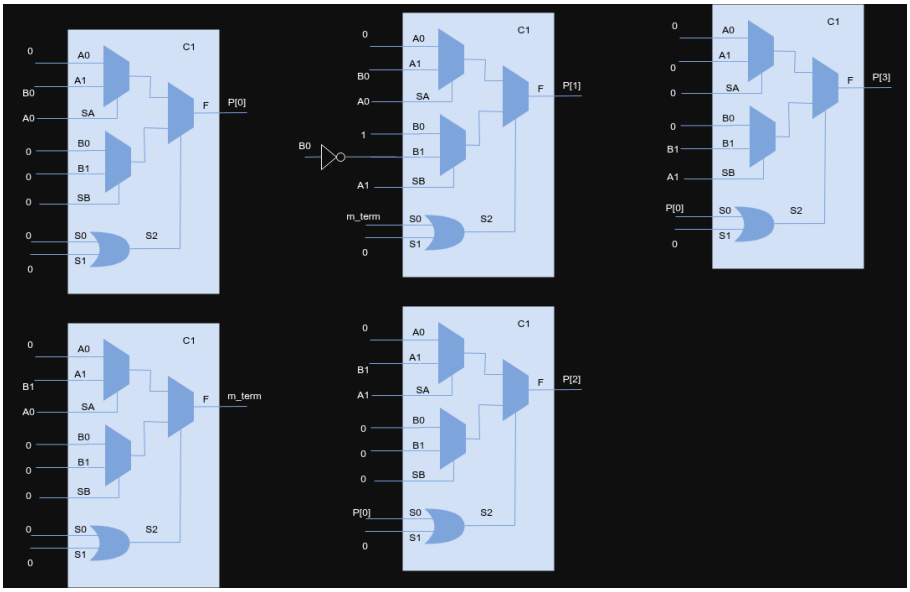


Functions:

Testbenches code in modules_code/TestBenches/functions_tb.v .



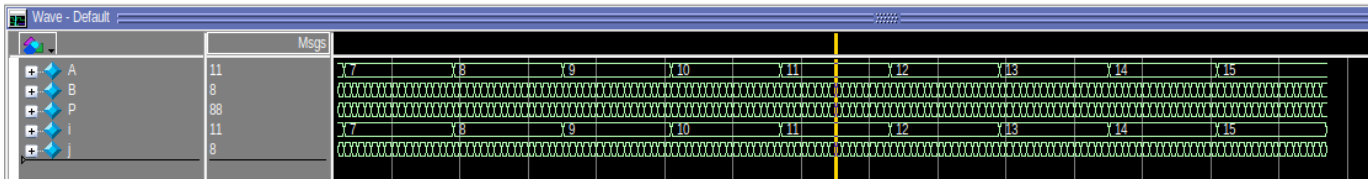
Multiplier 2x2:



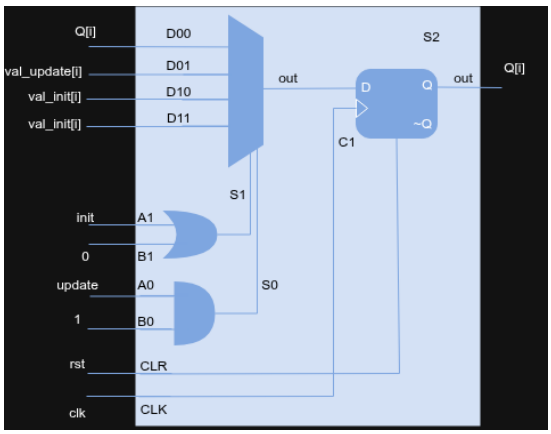
Multiplier 4x4:

Designed by 2x2 multiplier, HA and FA. the codes are in **modules_code/CoreModules/Multiplier.v**

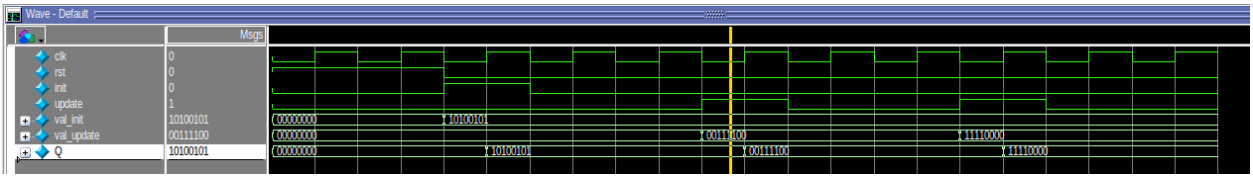
Testbenches code in modules_code/TestBenches/Multiplier_TB.v



Mux Reg:

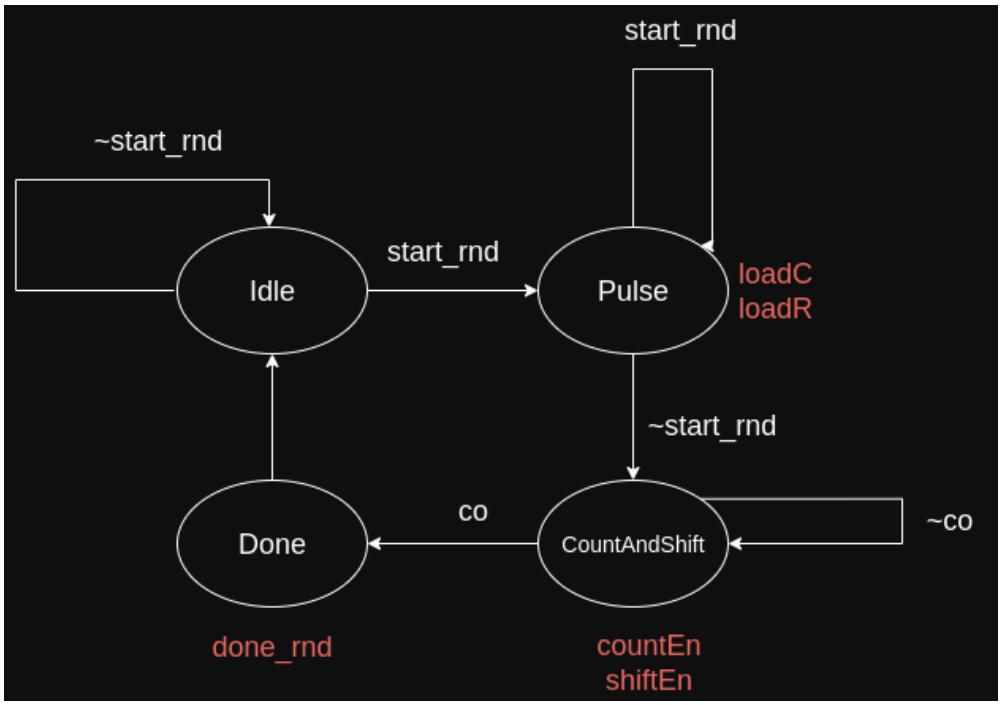


Testbenches code in modules_code/TestBenches/mux_reg_tb.v

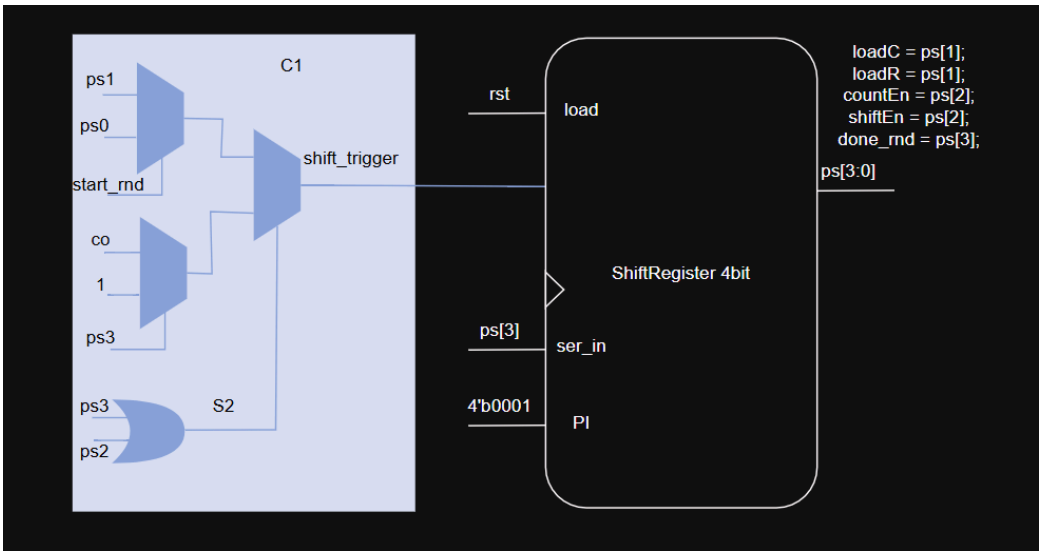


Random Generator

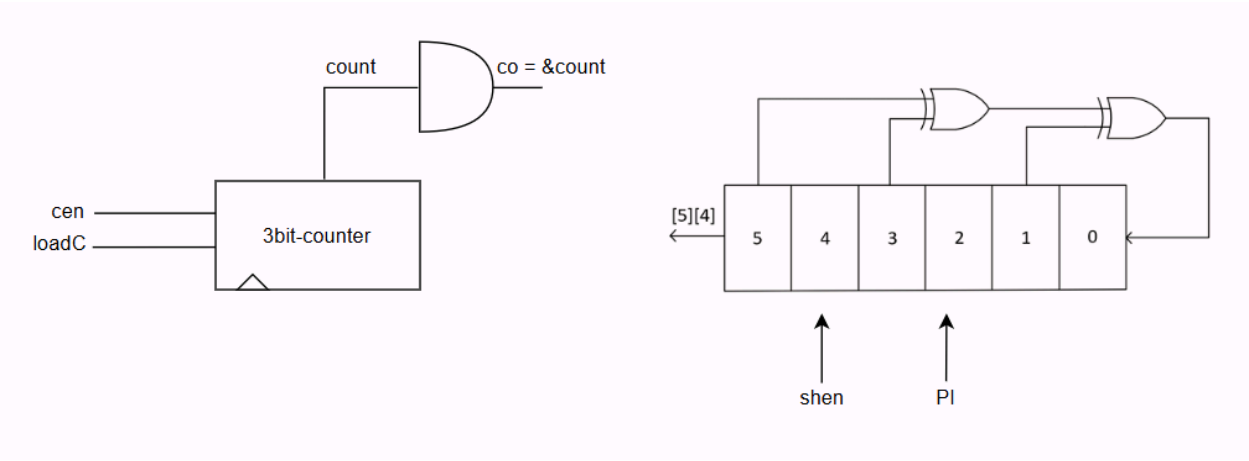
Controller:



Implementation with one hot method:

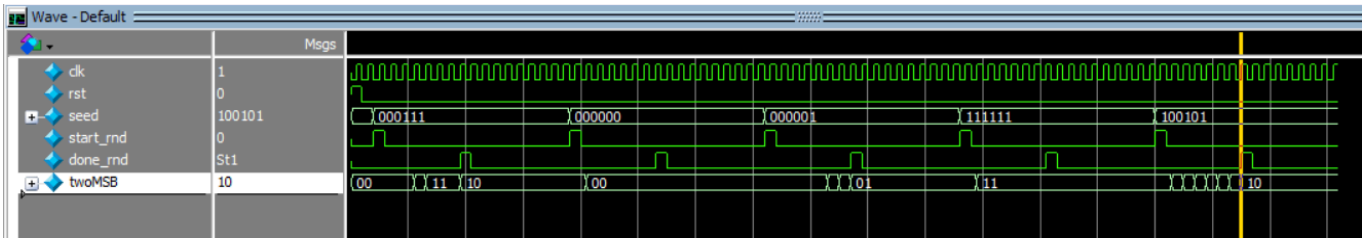


Data Path:



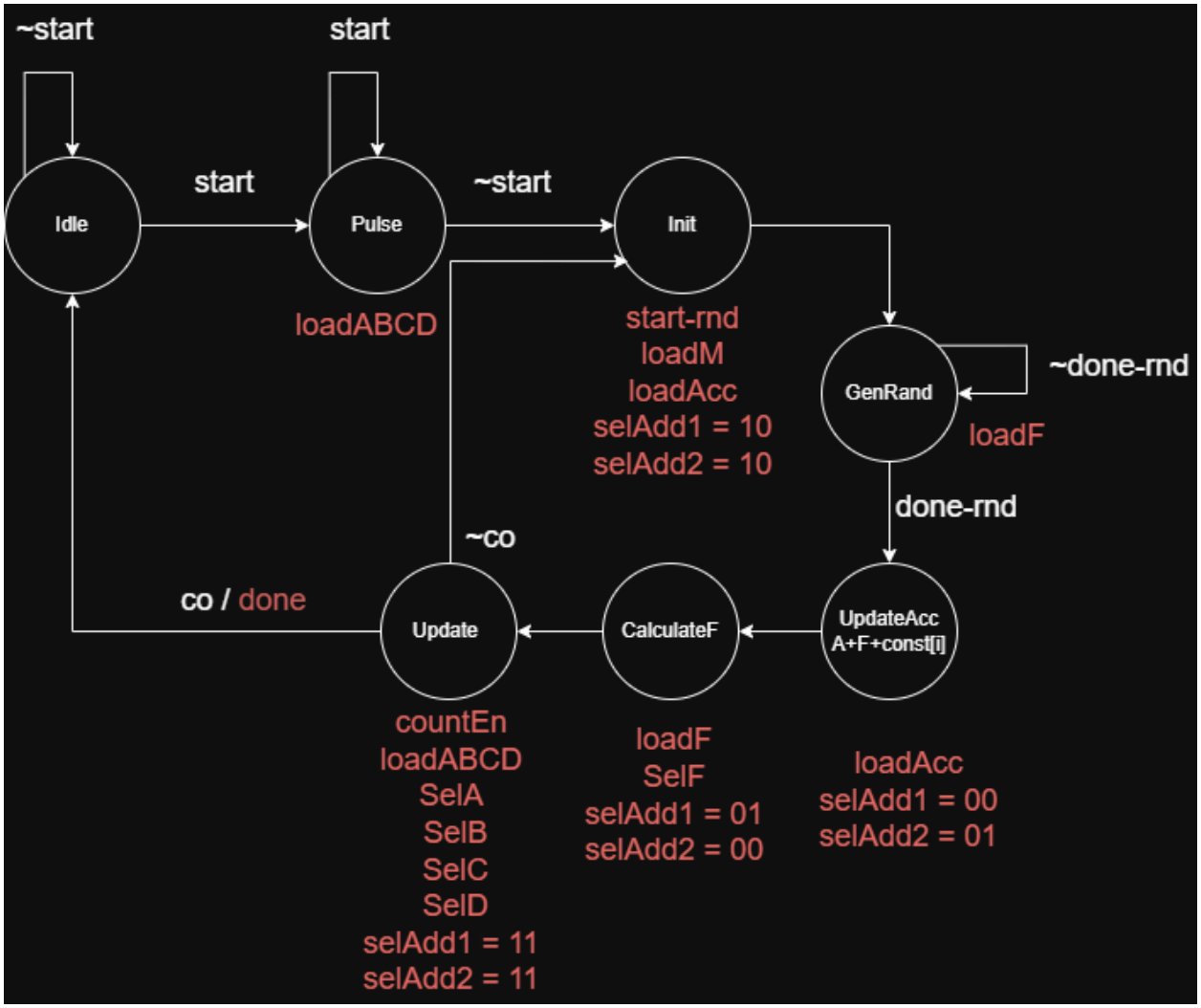
Test Bench:

Testbenches code in modules_code/TestBenches/rand_gen_tb.v

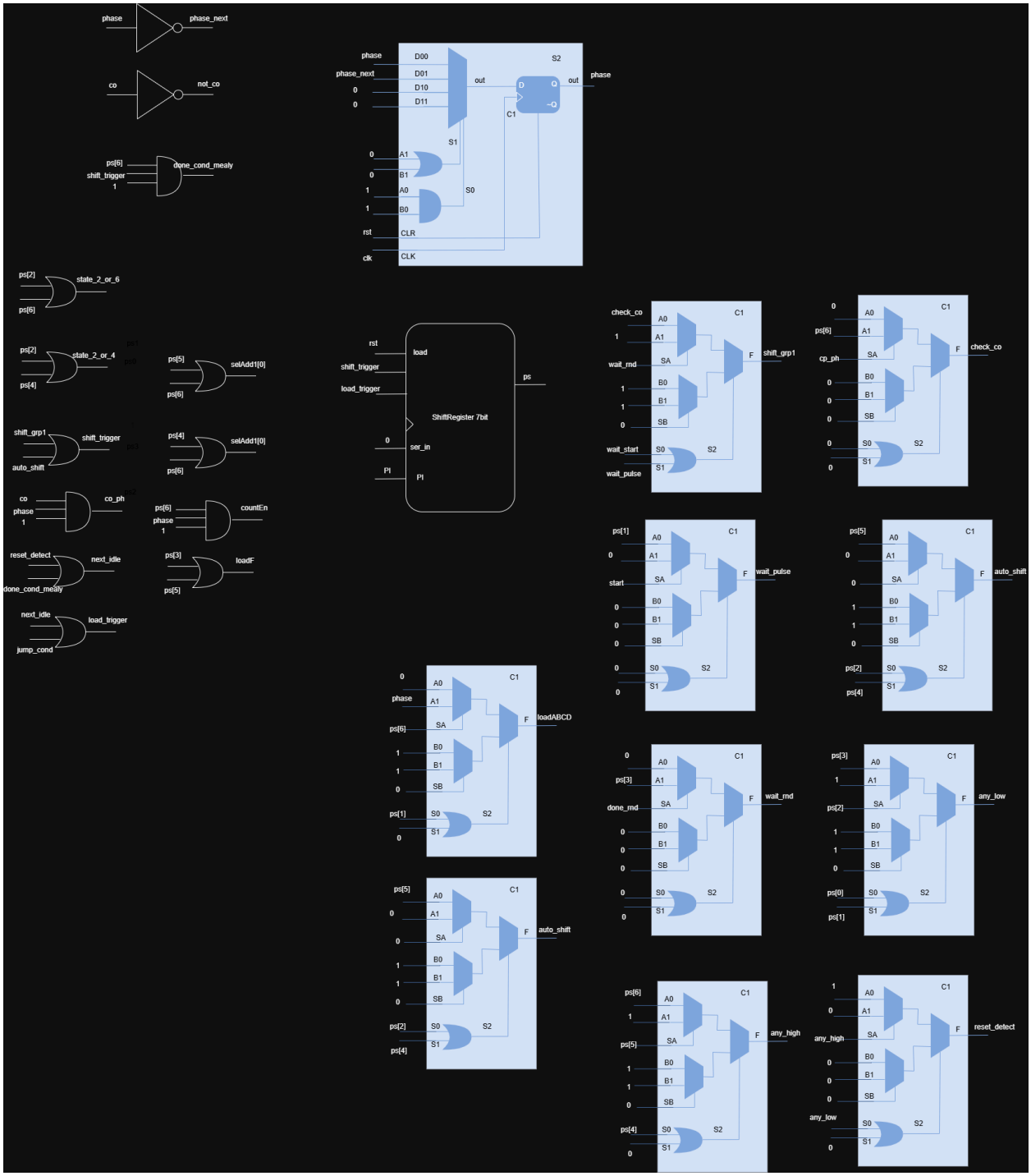


Hash Generator

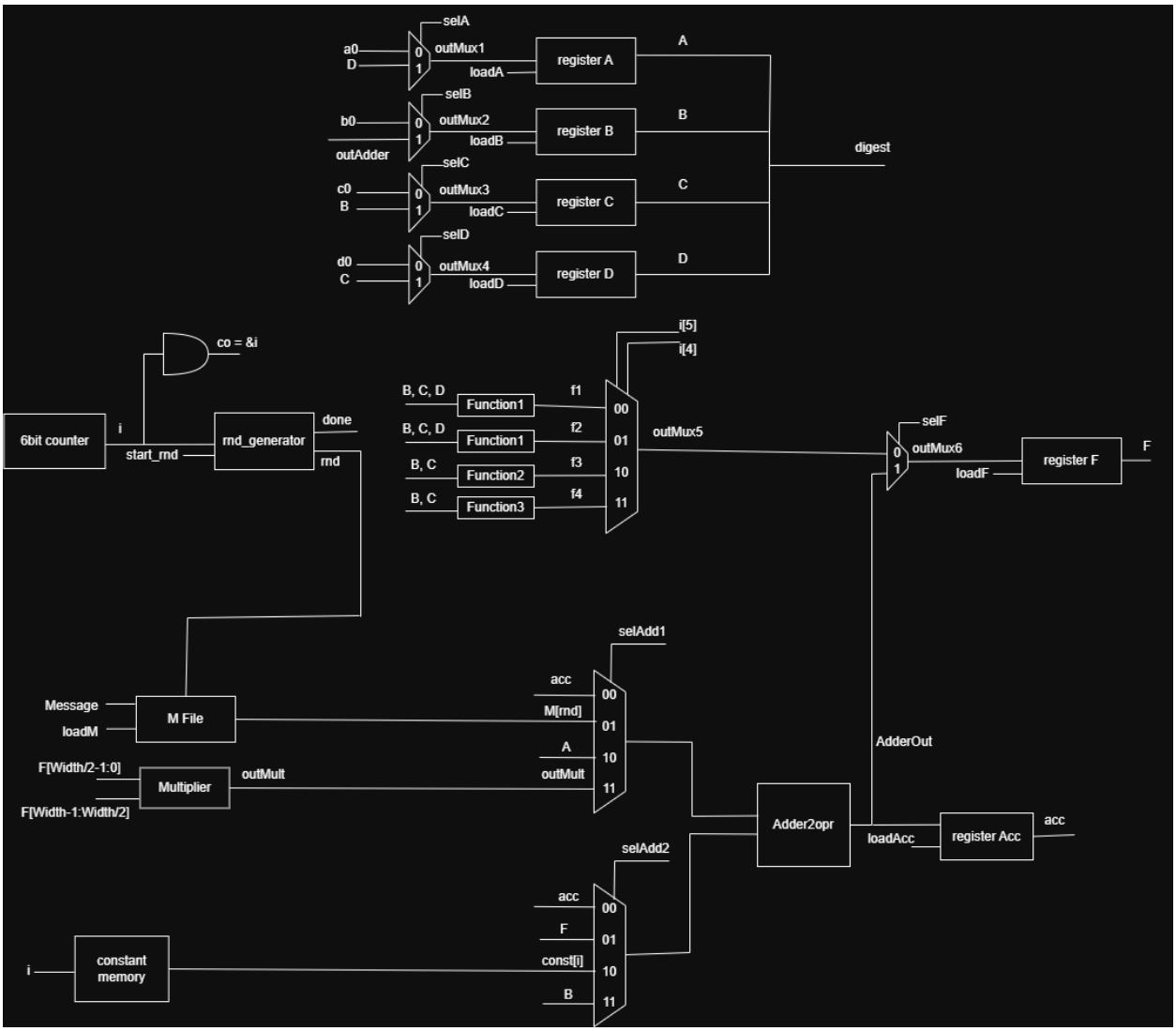
Controller:



Implementation with one hot method:

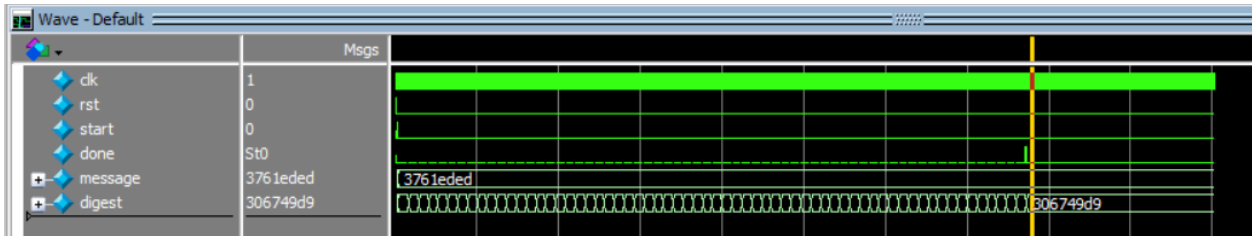


Data Path:


selA
a0
D
outMux1
loadA
register A
A
selB
b0
outAdder
outMux2
loadB
register B
B
digest
selC
c0
B
outMux3
loadC
register C
C
selD
d0
C
outMux4
loadD
register D
D
co = &i
6bit counter
i
start_rnd
rnd_generator
done
rnd
B, C, D
Function1
f1
B, C, D
Function1
f2
B, C
Function2
f3
B, C
Function3
f4
i[5]
i[4]
00
01
10
11
outMux5
selF
outMux6
loadF
register F
F
selAdd1
Message
loadM
M File
acc
M[rnd]
A
outMult
F[Width/2-1:0]
F[Width-1:Width/2]
Multiplier
outMult
AdderOut
Adder2opr
loadAcc
register Acc
acc
selAdd2
acc
F
const[i]
B
i
constant memory

Test Bench:

Testbenches code in modules_code/TestBenches/hash_gen_tb.v



Total number of gates used:

3421